TalentRank AI

Explainable AI Candidate Discovery Engine — turning a job description and a candidate pool into a transparent, evidence-backed shortlist.

Data & AI Challenge | Intelligent Candidate Discovery | INDIA RUNS Hackathon

PROBLEM

Recruiters can't review candidate pools deeply enough

Traditional applicant-tracking workflows lean on keyword filters. That approach misses semantically relevant candidates, over-ranks shallow keyword matches, and gives recruiters little basis to defend a shortlist decision.

Keyword filters are brittle

A candidate who says "led a team" instead of "management experience" gets skipped, even with the right background.

Rankings aren't explainable

Recruiters need evidence — matched skills, experience fit, gaps — not a black-box score.

Sensitive data creeps into scoring

Name, gender, age, and similar fields must never influence a ranking decision.

WHAT WE BUILT

A recruiter decision-support workflow, not a black box

1. JD Intelligence

Rule-based extraction of role, seniority, must-have / nice-to-have skills, responsibilities, domain, and experience range.

2. Privacy-safe profiles

Candidate and job text built only from approved ranking fields — sensitive attributes are never added.

3. Hybrid retrieval

TF-IDF + Sentence-Transformer (MiniLM) semantic similarity + skill coverage narrow the pool to a top-50 shortlist.

4. Optional CrossEncoder rerank

Pairwise relevance scoring over the top-50 for higher-fidelity ordering when latency allows.

5. Explanation cards

Every ranked candidate gets a fit summary, matched/missing skills, evidence, and a risk statement.

6. Responsible AI report

Every run writes an audit trail of used features, excluded fields, and the decision-support disclaimer.

WHY WE BUILT IT THIS WAY

Design choices, and the reasoning behind them

Hybrid over pure semantic

Semantic-only retrieval misses exact skill/tool matches recruiters expect to see (e.g. "AWS"). TF-IDF keeps lexical precision; embeddings add meaning. Combining both outperforms either alone on a small, noisy candidate pool.

Retrieve-then-rerank, not rerank-everything

CrossEncoder scoring is accurate but slow on CPU. Restricting it to the top-50 hybrid shortlist keeps latency practical while still getting the most accurate model where it matters most.

Explainability as a first-class output

A ranked list without justification isn't useful to a recruiter who has to defend a shortlist decision — so every candidate carries structured evidence, not just a number.

Fairness enforced in code, not policy

`src/fairness.py` excludes name, gender, religion, caste, age, DOB, photo, marital status, and address from profile text and scoring — structurally, not by convention.

Streamlit for the demo surface

Fast to build, easy for judges to run locally, and good enough for a recruiter-facing decision-support tool at hackathon scope.

HOW IT WORKS

End-to-end pipeline

1 Parse job description → role, seniority, skills, experience →

2 Build privacy-safe candidate & job profile text →

3 TF-IDF + MiniLM embeddings → hybrid top-50 retrieval →

4 Optional CrossEncoder pairwise rerank

5 Add skill / experience / activity evidence →

6 Final weighted score + explanation card →

7 Export CSV / JSON / XLSX + Responsible AI report

Scoring formula (CrossEncoder enabled)

```
final_score = 0.30 × cross_encoder_score + 0.20 × skill_match_score + 0.20 × semantic_score
      + 0.15 × tfidf_score + 0.10 × experience_match_score + 0.05 × activity_score
```

When CrossEncoder is disabled, its 30% weight redistributes equally to semantic similarity and skill coverage.

TECH STACK & EVALUATION

Built with a standard, auditable ML stack

Python 3.11 · pandas / NumPy · scikit-learn TF-IDF · Sentence-Transformers MiniLM-L6-v2 · CrossEncoder ms-marco-MiniLM-L-6-v2 · Streamlit · pytest

Evaluation metrics reported by the pipeline

P@5 / P@10
Precision at top-K

Recall@10
with relevance labels

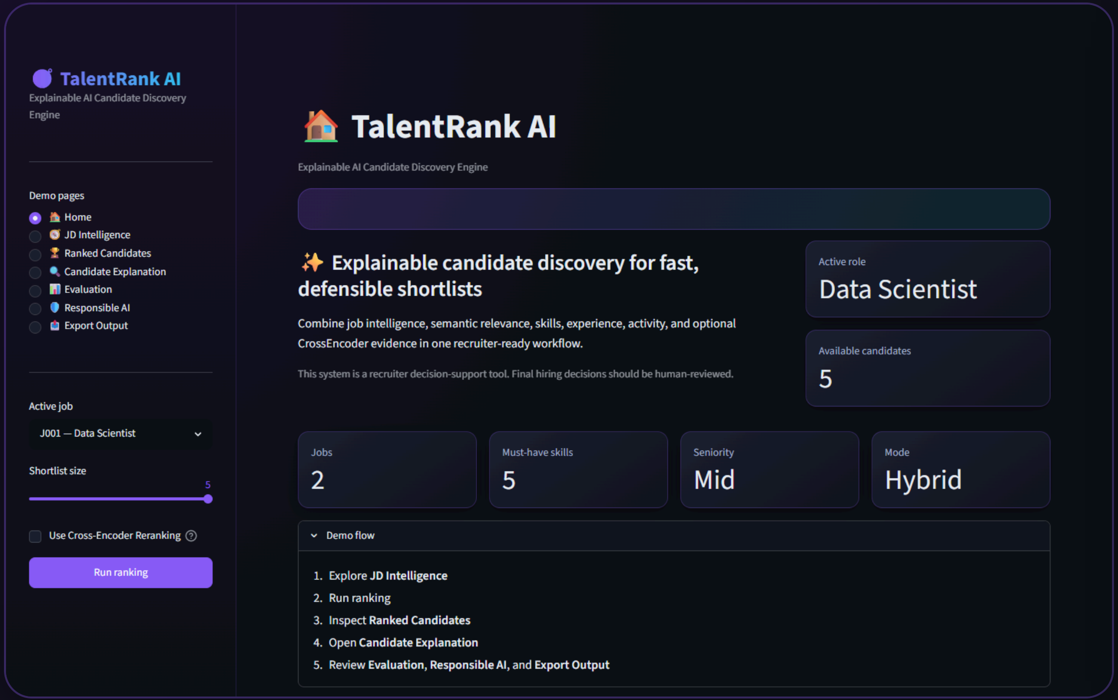
NDCG@10
ranked relevance quality

MRR
first-relevant-hit rank

Without labels: average top-10 skill coverage, semantic score, experience match, and ranking latency — plus an ablation comparing TF-IDF only, Semantic only, Hybrid, and Hybrid + CrossEncoder.

DASHBOARD

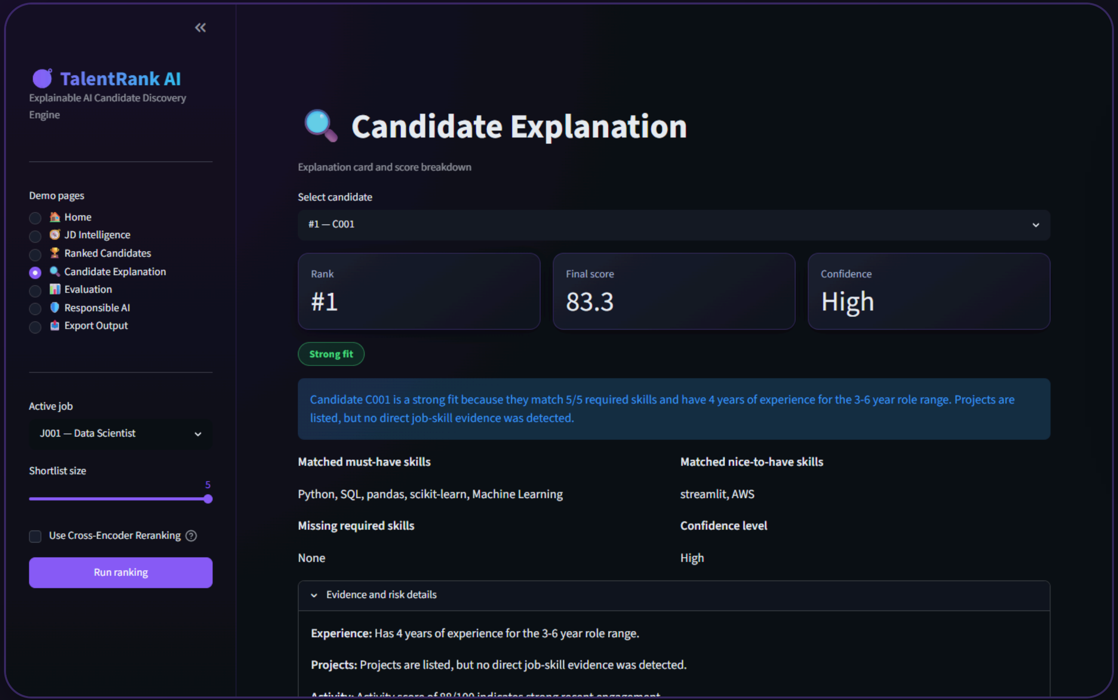
Home & Ranked Candidates

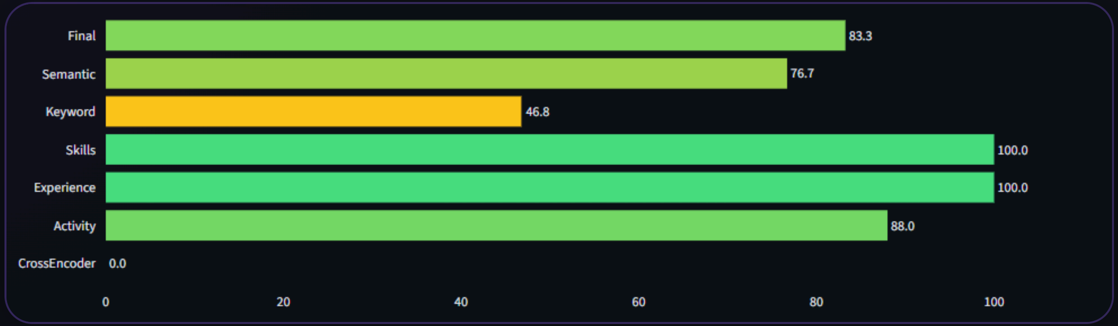


rank	candidate_id	final_score	cross_encoder_score	semantic_score	tfidf_score	skill_match_score	experience_match_score	activity_score
1	C001	83.26	None	76.7	46.78	100	100	88
2	C002	61.01	None	69.37	12.88	60	100	76
3	C004	45.49	None	52.89	18.06	40	66.67	72
4	C005	44.77	None	65.15	9.77	20	100	70
5	C003	25.24	None	31.96	0	0	100	81

DASHBOARD

Candidate Explanation & Score Breakdown





LIMITATIONS & WHAT'S NEXT

Honest about scope, clear on the roadmap

Current limitations

- Sample data is small and not representative of production hiring volume
- Skill extraction is rule-based and depends on taxonomy coverage
- Scores are decision-support signals, not job-performance predictions
- CrossEncoder reranking adds noticeable CPU latency

Future improvements

- Expand the skill ontology with role/industry-specific aliases
- Multilingual resume and JD support
- Recruiter feedback loop for calibrated relevance labels
- Vector-store retrieval for larger candidate pools
- Auth, audit trails, and RBAC for production deployment

This system is a recruiter decision-support tool. Final hiring decisions should always be human-reviewed.